

Deploying ML Models on AWS ECS and EKS

A Step-by-Step Container Orchestration Guide

Prabal

April 20, 2026

Contents

1	Introduction	2
2	ECS vs. EKS — Choosing	2
3	Common Prerequisites	3
4	Building the Inference Image	3
5	Path A — Deploy on ECS (Fargate)	4
5.1	Step A1 — Create the ECS Cluster	4
5.2	Step A2 — IAM Roles	4
5.3	Step A3 — Task Definition	4
5.4	Step A4 — Application Load Balancer	5
5.5	Step A5 — Create the Service	5
5.6	Step A6 — Service Autoscaling	6
5.7	Step A7 — Rolling Updates	6
6	Path B — Deploy on EKS	6
6.1	Step B1 — Create the EKS Cluster	7
6.2	Step B2 — GPU Support	7
6.3	Step B3 — IAM Roles for Service Accounts (IRSA)	7
6.4	Step B4 — Deployment Manifest	7
6.5	Step B5 — Service and Ingress	8
6.6	Step B6 — Horizontal Pod Autoscaler	9
6.7	Step B7 — Node Autoscaling	9
6.8	Step B8 — Purpose-Built ML Serving on EKS	9
7	GPU and Accelerator Notes	10
8	Model Weight Storage Strategies	10
9	Observability	10
10	Cost Considerations	11
11	Production Best Practices	11
12	Troubleshooting Checklist	11
13	Summary	12

1 Introduction

When your ML workload outgrows SageMaker’s managed abstractions or Lambda’s size and GPU limitations, AWS offers two container orchestrators for running custom inference stacks at scale:

- **Amazon ECS** (Elastic Container Service) — AWS-native orchestrator. Simpler mental model, tight integration with the AWS ecosystem, and lower operational overhead.
- **Amazon EKS** (Elastic Kubernetes Service) — managed Kubernetes control plane. More portable, more powerful, and backed by the enormous Kubernetes ecosystem (Helm, Argo, KServe, Kubeflow), at the cost of higher complexity.

Both pull container images from **Amazon ECR** and can run on two underlying compute platforms:

- **Fargate** — serverless containers, no EC2 to manage. No GPU.
- **EC2** — you manage the instance fleet; required for GPU, Inferentia, spot, and custom kernels.

Note

When to choose ECS / EKS over SageMaker:

- You need a custom serving stack — Triton Inference Server, TorchServe, Ray Serve, vLLM, Text Generation Inference.
- You already run Kubernetes for the rest of your platform.
- You want fine-grained control over networking, sidecars, service mesh.
- Cost: EC2 reservations / spot can beat SageMaker list prices.
- You are serving LLMs or diffusion models and need exotic GPU topologies (multi-GPU, tensor parallelism).

2 ECS vs. EKS — Choosing

Dimension	ECS	EKS
Control plane cost	Free	\$0.10/hour/cluster (~\$73/mo)
Learning curve	Low	High (Kubernetes)
Portability	AWS only	Runs anywhere Kubernetes runs
Ecosystem	AWS-centric	Helm, Argo, KServe, Kubeflow, Istio
Autoscaling	Service + cluster autoscaling	HPA, VPA, KEDA, Karpenter
Best for	Small/medium teams, AWS-only shops	Platform teams, multi-tenant, advanced ML stacks

Tip

Rule of thumb: Start with ECS on Fargate. Move to EKS only when you hit a specific need Kubernetes uniquely solves — Helm-based OSS inference stacks (KServe, Ray Serve, vLLM), multi-model GPU sharing, or an existing platform engineering investment.

3 Common Prerequisites

1. AWS CLI configured, Docker installed and running.
2. A VPC with at least two subnets across different Availability Zones.
3. An ECR repository holding your inference image (see §4).
4. A trained model artifact (baked in image *or* stored in S3 / EFS / FSx for Lustre).
5. IAM permissions to create ECS clusters / EKS clusters, task roles, load balancers, autoscaling policies.

4 Building the Inference Image

Unlike Lambda, there is no AWS-provided runtime wrapper — your container simply listens on an HTTP port and responds to health checks. A minimal FastAPI + PyTorch image:

```
FROM python:3.11-slim

WORKDIR /app

# System deps (if needed for your model)
RUN apt-get update && apt-get install -y --no-install-recommends \
    build-essential && rm -rf /var/lib/apt/lists/*

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY app/ ./app/
COPY model/ ./model/

EXPOSE 8080
CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8080", \
    "--workers", "1"]
```

Minimal FastAPI app (app/main.py):

```
1 from fastapi import FastAPI, HTTPException
2 from pydantic import BaseModel
3 import torch
4
5 app = FastAPI()
6 MODEL = torch.load("model/model.pt", map_location="cpu")
7 MODEL.eval()
8
9 class Request(BaseModel):
10     inputs: list
11
12 @app.get("/health")
13 def health():
14     return {"status": "ok"}
15
16 @app.post("/predict")
17 def predict(req: Request):
18     try:
19         x = torch.tensor(req.inputs, dtype=torch.float32)
20         with torch.inference_mode():
21             y = MODEL(x)
22         return {"predictions": y.tolist()}
23     except Exception as e:
24         raise HTTPException(status_code=500, detail=str(e))
```

Push to ECR (same steps as Lambda doc):

```
export AWS_REGION=us-east-1
export ACCOUNT_ID=$(aws sts get-caller-identity --query Account --output text)
export REPO=ml-inference

aws ecr create-repository --repository-name $REPO --region $AWS_REGION

aws ecr get-login-password --region $AWS_REGION | \
  docker login --username AWS \
    --password-stdin $ACCOUNT_ID.dkr.ecr.$AWS_REGION.amazonaws.com

docker build --platform linux/amd64 -t $REPO:v1 .
docker tag $REPO:v1 $ACCOUNT_ID.dkr.ecr.$AWS_REGION.amazonaws.com/$REPO:v1
docker push $ACCOUNT_ID.dkr.ecr.$AWS_REGION.amazonaws.com/$REPO:v1
```

Tip

For production-grade model servers, skip DIY FastAPI and use a purpose-built server: **NVIDIA Triton** (multi-framework, dynamic batching, model versioning), **TorchServe**, **TensorFlow Serving**, **vLLM** (LLMs), or **Text Generation Inference**. They handle batching, multi-model hosting, and metrics far better than a handwritten server.

5 Path A — Deploy on ECS (Fargate)

ECS models a workload as:

- **Task Definition** — the “recipe”: image URI, CPU/memory, env vars, ports, logging.
- **Service** — long-running controller that keeps N tasks healthy behind a load balancer.
- **Cluster** — logical grouping of services and tasks.

5.1 Step A1 — Create the ECS Cluster

```
aws ecs create-cluster \
  --cluster-name ml-cluster \
  --capacity-providers FARGATE FARGATE_SPOT \
  --default-capacity-provider-strategy \
    capacityProvider=FARGATE,weight=1
```

5.2 Step A2 — IAM Roles

You need two roles:

- **Task Execution Role** — used by ECS itself to pull from ECR and write logs to CloudWatch. Attach `AmazonECSTaskExecutionRolePolicy`.
- **Task Role** — assumed by the *application* inside the container. Grant it whatever AWS permissions your inference code needs (e.g., `s3:GetObject` on the model bucket).

5.3 Step A3 — Task Definition

task-def.json:

```
{
  "family": "ml-inference",
  "networkMode": "awsvpc",
  "requiresCompatibilities": ["FARGATE"],
  "cpu": "2048",
  "memory": "8192",
```

```

"executionRoleArn": "arn:aws:iam::ACCOUNT:role/ecsTaskExecutionRole",
"taskRoleArn":      "arn:aws:iam::ACCOUNT:role/ml-inference-task-role",
"containerDefinitions": [{
  "name": "ml",
  "image": "ACCOUNT.dkr.ecr.us-east-1.amazonaws.com/ml-inference:v1",
  "essential": true,
  "portMappings": [{ "containerPort": 8080, "protocol": "tcp" }],
  "healthCheck": {
    "command": ["CMD-SHELL", "curl -f http://localhost:8080/health || exit 1"],
    "interval": 30, "timeout": 5, "retries": 3, "startPeriod": 30
  },
  "logConfiguration": {
    "logDriver": "awslogs",
    "options": {
      "awslogs-group": "/ecs/ml-inference",
      "awslogs-region": "us-east-1",
      "awslogs-stream-prefix": "ecs"
    }
  }
}]
}

```

Register it:

```
aws ecs register-task-definition --cli-input-json file://task-def.json
```

Important

Fargate CPU/memory combinations are *not* arbitrary. Valid pairs include 0.25 vCPU / 0.5–2 GB, 2 vCPU / 4–16 GB, 16 vCPU / 32–120 GB, etc. Check the Fargate task sizing table before picking values or the task will fail to register.

5.4 Step A4 — Application Load Balancer

Create an internet-facing ALB in your public subnets, then a target group of type `ip` (required for Fargate with `awsvpc` networking):

```

aws elbv2 create-target-group \
  --name ml-tg \
  --protocol HTTP --port 8080 \
  --vpc-id vpc-XXXXXXX \
  --target-type ip \
  --health-check-path /health \
  --health-check-interval-seconds 30

```

Attach a listener forwarding `:80` (or `:443` with an ACM cert) to the target group.

5.5 Step A5 — Create the Service

```

aws ecs create-service \
  --cluster ml-cluster \
  --service-name ml-inference \
  --task-definition ml-inference \
  --desired-count 2 \
  --launch-type FARGATE \
  --network-configuration "awsvpcConfiguration={
    subnets=[subnet-A,subnet-B],
    securityGroups=[sg-XXXX],
    assignPublicIp=DISABLED}" \

```

```
--load-balancers "targetGroupArn=arn:...:targetgroup/ml-tg/..., \
  containerName=ml, containerPort=8080" \
--deployment-configuration \
  "minimumHealthyPercent=100, maximumPercent=200"
```

ECS will launch two tasks, register them with the ALB, and replace any that fail health checks.

5.6 Step A6 — Service Autoscaling

Register the service as a scalable target and attach a target-tracking policy:

```
aws application-autoscaling register-scalable-target \
  --service-namespace ecs \
  --resource-id service/ml-cluster/ml-inference \
  --scalable-dimension ecs:service:DesiredCount \
  --min-capacity 2 --max-capacity 20

aws application-autoscaling put-scaling-policy \
  --policy-name ml-cpu-scale \
  --service-namespace ecs \
  --resource-id service/ml-cluster/ml-inference \
  --scalable-dimension ecs:service:DesiredCount \
  --policy-type TargetTrackingScaling \
  --target-tracking-scaling-policy-configuration file://scale.json
```

Where `scale.json` targets 60% CPU utilization with cooldowns.

Tip

For latency-sensitive services, scale on `ALBRequestCountPerTarget` or a custom CloudWatch metric (e.g., queue depth) rather than CPU — CPU reacts too slowly and often stays flat until the service is already saturated.

5.7 Step A7 — Rolling Updates

Push a new image tag, register a new task-definition revision, then:

```
aws ecs update-service \
  --cluster ml-cluster \
  --service ml-inference \
  --task-definition ml-inference:2 \
  --force-new-deployment
```

With `minimumHealthyPercent=100`, `maximumPercent=200`, ECS launches new tasks alongside old ones and drains the old ones only after new ones are healthy (blue/green style).

For canary / linear / all-at-once with automatic rollback, switch the deployment controller to `CODE_DEPLOY` and configure CodeDeploy.

6 Path B — Deploy on EKS

EKS is managed Kubernetes. The ML-relevant objects are:

- **Deployment** — declarative pod template + replica count.
- **Service** — stable in-cluster IP / DNS.
- **Ingress / ALB Ingress** — external HTTPS entry point.
- **HPA** — Horizontal Pod Autoscaler, scales pods.
- **Karpenter / Cluster Autoscaler** — scales nodes.

6.1 Step B1 — Create the EKS Cluster

The easiest path is `eksctl`:

```
eksctl create cluster \
  --name ml-eks \
  --region us-east-1 \
  --version 1.29 \
  --nodegroup-name gpu-nodes \
  --node-type g5.xlarge \
  --nodes 2 --nodes-min 1 --nodes-max 10 \
  --with-oidc \
  --ssh-access --ssh-public-key my-key
```

For Fargate-only (CPU, no GPU):

```
eksctl create cluster \
  --name ml-eks \
  --fargate \
  --region us-east-1
```

Important

The EKS control plane costs **\$0.10/hour** (~\$73/month) *per cluster*, on top of node costs. Do not spin up clusters casually for dev. Share clusters across teams where possible.

6.2 Step B2 — GPU Support

For GPU nodes (e.g., g5, p4d, p5), install the NVIDIA device plugin DaemonSet:

```
kubectl apply -f \
  https://raw.githubusercontent.com/NVIDIA/k8s-device-plugin/v0.14.0/nvidia-
  device-plugin.yml
```

Pods then request GPUs via `resources.limits["nvidia.com/gpu"]`: 1.

6.3 Step B3 — IAM Roles for Service Accounts (IRSA)

IRSA maps a Kubernetes ServiceAccount to an IAM role, so only pods running as that SA get those AWS permissions — the least-privilege pattern on EKS.

```
eksctl create iamserviceaccount \
  --cluster ml-eks \
  --name ml-inference-sa \
  --namespace ml \
  --attach-policy-arn arn:aws:iam::aws:policy/AmazonS3ReadOnlyAccess \
  --approve
```

6.4 Step B4 — Deployment Manifest

`deployment.yaml`:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: ml-inference
  namespace: ml
spec:
  replicas: 2
  strategy:
    type: RollingUpdate
```

```

rollingUpdate:
  maxSurge: 1
  maxUnavailable: 0
selector:
  matchLabels:
    app: ml-inference
template:
  metadata:
    labels:
      app: ml-inference
  spec:
    serviceAccountName: ml-inference-sa
    containers:
    - name: ml
      image: ACCOUNT.dkr.ecr.us-east-1.amazonaws.com/ml-inference:v1
      ports:
      - containerPort: 8080
      resources:
        requests:
          cpu: "1"
          memory: "4Gi"
          nvidia.com/gpu: "1"      # remove for CPU-only
        limits:
          cpu: "2"
          memory: "8Gi"
          nvidia.com/gpu: "1"
      readinessProbe:
        httpGet: {path: /health, port: 8080}
        initialDelaySeconds: 30
        periodSeconds: 10
      livenessProbe:
        httpGet: {path: /health, port: 8080}
        initialDelaySeconds: 60
        periodSeconds: 30

```

6.5 Step B5 — Service and Ingress

service.yaml:

```

apiVersion: v1
kind: Service
metadata:
  name: ml-inference
  namespace: ml
spec:
  type: ClusterIP
  selector:
    app: ml-inference
  ports:
  - port: 80
    targetPort: 8080
    protocol: TCP

```

External exposure via the **AWS Load Balancer Controller** (install via Helm once per cluster), which provisions a real AWS ALB from an Ingress:

```

apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: ml-inference
  namespace: ml
  annotations:
    kubernetes.io/ingress.class: alb

```

```

alb.ingress.kubernetes.io/scheme: internet-facing
alb.ingress.kubernetes.io/target-type: ip
alb.ingress.kubernetes.io/healthcheck-path: /health
spec:
  rules:
  - http:
      paths:
      - path: /
        pathType: Prefix
        backend:
          service:
            name: ml-inference
            port:
              number: 80

```

Apply everything:

```

kubectl create namespace ml
kubectl apply -f deployment.yaml
kubectl apply -f service.yaml
kubectl apply -f ingress.yaml

```

6.6 Step B6 — Horizontal Pod Autoscaler

```

apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: ml-inference
  namespace: ml
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: ml-inference
  minReplicas: 2
  maxReplicas: 20
  metrics:
  - type: Resource
    resource:
      name: cpu
      target:
        type: Utilization
        averageUtilization: 60

```

For queue-depth-based scaling, custom metrics, or scale-to-zero, install **KEDA** — it supports SQS, Kafka, Prometheus, CloudWatch, and many other external triggers.

6.7 Step B7 — Node Autoscaling

Two options:

- **Cluster Autoscaler** — reactive, per-node-group, simple.
- **Karpenter** — modern AWS-native autoscaler; picks the cheapest suitable instance type dynamically (great for mixed CPU/GPU fleets and spot). Recommended for new clusters.

6.8 Step B8 — Purpose-Built ML Serving on EKS

Rather than writing your own Deployments, many teams deploy pre-built inference platforms on EKS:

- **KServe** — standard CRDs (`InferenceService`) with canary, autoscaling, explainability, batching. The de-facto model serving layer on Kubernetes.
- **Ray Serve** — Python-first, great for composition and multi-model pipelines.
- **Seldon Core** — enterprise-grade inference graphs.
- **NVIDIA Triton** on EKS — best-in-class GPU utilization, dynamic batching, multi-framework.
- **vLLM / TGI** — LLM-specific servers with paged attention and continuous batching.

7 GPU and Accelerator Notes

Instance	Accelerator	Use case
g4dn.*	NVIDIA T4 (16 GB)	Low-cost inference, small-to-medium DL
g5.*	NVIDIA A10G (24 GB)	Mid-sized LLMs ($\leq 13B$), SD, CV
p4d.*	8× A100 (40 GB)	Large LLMs, training, tensor parallel
p5.*	8× H100 (80 GB)	Frontier models, long context
inf1.*	AWS Inferentia 1	Cheapest \$/inference for supported models
inf2.*	AWS Inferentia 2	Larger models, generative AI
trn1.*	AWS Trainium	Training (occasionally inference)

Tip

For steady-state LLM serving, `inf2` with the Neuron SDK is often 30–50% cheaper than equivalent NVIDIA GPUs — at the cost of a narrower model-support matrix and a model-compilation step.

8 Model Weight Storage Strategies

1. **Baked in image** — simplest, deterministic. Image rebuilds on every model update. Good for $\leq$ few GB.
2. **S3 at startup** — image stays small; cold start downloads weights to local disk. Typical for 5–50 GB models.
3. **EFS mount** — shared read-only filesystem across pods. Great for multi-model endpoints. Medium throughput.
4. **FSx for Lustre** — high-throughput parallel filesystem backed by S3. Standard for large-scale GPU inference (LLMs) where you need many GB loaded fast across many pods.

9 Observability

ECS. CloudWatch Container Insights (one flag at cluster creation), plus standard CloudWatch Logs from the `awslogs` driver. AWS Distro for OpenTelemetry (ADOT) for traces and custom metrics.

EKS. Install the standard Kubernetes observability stack:

- **Prometheus** (AMP — Amazon Managed Prometheus) + **Grafana** (AMG) for metrics.
- **Fluent Bit** → CloudWatch Logs / OpenSearch.
- **AWS Distro for OpenTelemetry** for traces.
- **DCGM Exporter** for GPU metrics (utilization, memory, temperature, throttling).

10 Cost Considerations

- **Spot instances** — up to 90% off on-demand; EKS + Karpenter handles interruption gracefully. Great for batch or stateless inference replicas.
- **Savings Plans** for steady 24/7 workloads (1-year or 3-year commitment, 30–50% savings).
- **Right-size continuously** — most teams over-provision memory by 2–3×. Use Container Insights or Kubecost to find slack.
- **Scale to zero** overnight / off-hours via KEDA cron scalers or scheduled ECS service updates.
- **Share GPUs** via MIG (A100/H100) or NVIDIA Time-Slicing for small models — up to 7× density on a single card.

11 Production Best Practices

1. **IaC everything** — Terraform, CDK, Pulumi. Never click-ops a production cluster.
2. **GitOps on EKS** — Argo CD or Flux for declarative, auditable deployments from a Git repo.
3. **Pod Disruption Budgets** on EKS so voluntary disruptions (node upgrades) don't take down all replicas.
4. **Multi-AZ** — spread replicas across ≥ 2 AZs with topology spread constraints / ECS placement strategies.
5. **Warm replicas always ≥ 2** — avoid single-point-of-failure on rolling updates or AZ failures.
6. **Image scanning** — enable ECR scan-on-push; block critical CVEs at CI time.
7. **Dynamic batching** — if using Triton or a custom server, enable micro-batching; doubles or triples throughput on GPU.
8. **Separate training and inference clusters** — different scaling profiles, security posture, and cost lifecycles.

12 Troubleshooting Checklist

Symptom	Likely Cause / Fix
ECS task stuck in PENDING	Usually networking: the task cannot pull the image. Check that private subnets have a NAT Gateway <i>or</i> VPC endpoints for ECR + S3 + CloudWatch Logs.

<code>CannotPullContainerError</code>	Task execution role missing ECR permissions, <i>or</i> image is for the wrong architecture. Rebuild with <code>-platform linux/amd64</code> .
ALB targets unhealthy	Health check path returns non-2xx, or pod takes longer than the initial delay to start. Increase <code>initialDelaySeconds</code> / health check grace period.
EKS pod pending, <code>Insufficient nvidia.com/gpu</code>	No GPU nodes available or NVIDIA device plugin not installed. Check Karpenter / autoscaler node provisioning logs.
OOMKilled in Kubernetes	Memory request/limit too low — framework lazy-allocates. Set <code>limits.memory</code> generously or the kernel will reap the pod.
GPU utilization near 0%	Tiny batch sizes; enable dynamic batching in Triton / TGI, or raise client concurrency.
Image pull is slow on scale-up	Large image (≥ 5 GB). Use ECR <i>pull-through cache</i> + warm AMIs, or mount weights from EFS/FSx and keep the code image small.

13 Summary

ECS flow:

1. Build and push image to ECR.
2. Create ECS cluster.
3. Define task execution and task roles.
4. Register task definition (image, CPU/memory, ports, logging).
5. Create ALB + target group.
6. Create ECS Service with desired count, networking, load balancer.
7. Attach Application Auto Scaling policy.
8. Update via new task-definition revisions.

EKS flow:

1. Create cluster with `eksctl` (managed / Fargate / GPU node groups).
2. Install AWS Load Balancer Controller, NVIDIA device plugin, Karpenter, metrics-server.
3. Build and push image to ECR.
4. Define Deployment, Service, Ingress manifests.
5. Use IRSA for pod-level AWS permissions.
6. Attach HPA (or KEDA) for pod scaling; Karpenter for node scaling.
7. Optionally layer on KServe / Triton / Ray Serve for richer ML serving.

The decision. Choose **ECS** for simplicity and AWS-native workflows; choose **EKS** when you need Kubernetes' ecosystem, portability, or advanced serving primitives. In both cases, ECR is the image registry, Fargate is the no-ops compute option, and EC2 (GPU / Inferentia) is the high-performance compute option.